

Warehouse Robot Navigation Using Deep Reinforcement Learning

Aniruddha Datta, Sahasrangshu Roy, Niladri Banerjee

Department of Computer Science
Ramakrishna Mission Vivekananda Educational and Research Institute

Abstract

Autonomous navigation and task execution in warehouse environments is a fundamental problem in artificial intelligence and robotics. This project formulates a warehouse robot pickup-and-delivery task as a Markov Decision Process and solves it using Deep Reinforcement Learning. A custom Gymnasium environment inspired by the Taxi-v3 benchmark is developed with directional movement constraints representing warehouse obstacles. A Deep Q-Network (DQN) agent is trained to learn optimal navigation, pickup, and delivery behavior. Quantitative reward curves and qualitative visualizations demonstrate successful learning and policy execution.

Introduction

Problem Definition

Modern warehouses increasingly rely on autonomous mobile robots to perform repetitive logistics tasks such as item pickup, transportation, and delivery. The core problem addressed in this project is enabling a robot to autonomously navigate a structured warehouse environment, pick up a package from a designated location, and deliver it to a specified destination while respecting movement constraints and obstacles.

The task involves sequential decision-making under uncertainty, where the robot must choose actions that balance efficiency and correctness over time. Naive rule-based or shortest-path approaches are insufficient due to penalties for illegal actions and the presence of constrained navigation paths.

Relevance in Artificial Intelligence and Reinforcement Learning

This problem is a canonical example of a sequential decision-making task and is naturally modeled using Reinforcement Learning (RL). In RL, an agent learns optimal behavior through interaction with an environment by maximizing cumulative reward rather than relying on explicitly programmed rules.

Warehouse navigation and task execution are widely studied problems in AI, with applications in robotics, automated logistics, and industrial automation. The problem formulation closely aligns with Markov Decision Processes (MDPs), making it an ideal testbed for value-based RL algorithms such as Deep Q-Networks (DQN). Furthermore, incorporating obstacles and delayed rewards highlights the strengths of RL in learning long-horizon planning strategies.

Project Objectives

The primary objectives of this project are as follows:

- To design and implement a custom warehouse robot environment using the Gymnasium framework.
- To model the pickup-and-delivery task as a Markov Decision Process with clearly defined states, actions, rewards, and termination conditions.
- To train a Deep Q-Network agent capable of learning an effective navigation and task execution policy.
- To evaluate the learned policy using both quantitative metrics, such as reward curves, and qualitative visualizations through animated simulations.

Problem Formulation and Representation

This project formulates the warehouse robot navigation and delivery task as an Artificial Intelligence planning and decision-making problem. The task is modeled within the Reinforcement Learning framework as a Markov Decision Process, where an agent must learn an optimal policy through interaction with the environment.

Problem Domain

The problem domain corresponds to a path-planning and task-execution scenario in a structured warehouse environment. The warehouse is represented as a discrete 5×5 grid world containing four designated storage and delivery locations. At the beginning of each episode, a package is placed at one storage location, and a target delivery location is specified.

The robot operates under movement constraints imposed by directional walls between certain grid cells. These walls simulate real-world obstacles such as shelves or partitions and prevent movement in specific directions. As a result, the

robot must plan non-trivial navigation paths rather than relying on direct shortest-path movement.

State, Actions, and Constraints

State Representation. The environment state is encoded as a discrete integer representing a combination of: (i) the robot's grid position, (ii) the package status (at one of the four storage locations or carried by the robot), and (iii) the delivery destination. This encoding results in a total of $25 \times 5 \times 4 = 500$ possible states.

Actions / Operators. The robot can execute six discrete actions: move north, move south, move east, move west, pick up the package, and drop off the package. Movement actions attempt to change the robot's position by one grid cell, while pickup and drop-off actions modify the package state.

Constraints / Rules. Robot movement is constrained by grid boundaries and directional walls, which may block motion between adjacent cells. Pickup actions are valid only when the robot is located at the package's current position. Drop-off actions are valid only when the robot is carrying the package and is located at the designated delivery location. Invalid actions are penalized through negative rewards.

Goal Test. The goal of the task is achieved when the robot successfully drops off the package at the correct delivery location. Upon satisfying this condition, the episode terminates and a positive terminal reward is assigned.

Environment and Task Description

Since this is a reinforcement learning-based project, no external dataset is used. Instead, learning occurs entirely through interaction with a simulated environment. The warehouse navigation task is implemented as a custom environment using the Gymnasium framework and follows the standard agent-environment interaction loop.

Environment Overview

The environment represents a structured warehouse as a discrete 5×5 grid world. The robot operates within this grid and must complete a pickup-and-delivery task. Directional walls are placed between selected grid cells to model obstacles such as shelves and partitions, restricting movement in specific directions.

At the beginning of each episode, the robot starts at a random grid location. A package is placed at one of four predefined storage locations, and a target delivery location is randomly assigned. The episode continues until the package is successfully delivered or a maximum step limit is reached.

Observation Space

The observation space is discrete and encodes the complete environment state as a single integer. Each state represents a combination of: (i) the robot's current grid position, (ii) the package status (waiting at one of the storage locations or being carried), and (iii) the destination location.

This compact representation results in a finite state space of size:

$$25 \times 5 \times 4 = 500$$

possible states.

Action Space

The agent has access to six discrete actions:

- Move north
- Move south
- Move east
- Move west
- Pick up the package
- Drop off the package

Movement actions attempt to shift the robot by one grid cell, subject to boundary and wall constraints. Pickup and drop-off actions modify the package state and are only valid under specific conditions.

Reward Structure

The reward function is designed to encourage efficient task completion while penalizing invalid behavior:

- A step penalty of -1 for each time step
- A reward of $+20$ for successful delivery
- A penalty of -10 for illegal pickup or drop-off actions

This reward design promotes shorter paths, discourages invalid actions, and provides a clear learning signal for successful task completion.

Episode Termination

An episode terminates when the robot successfully delivers the package to the correct destination or when a predefined maximum number of steps is reached. Upon termination, the environment resets and a new episode begins.

Methodology

This section presents the reinforcement learning methodology adopted in this project, with emphasis on the mathematical foundations of Deep Q-Networks (DQN). All equations are introduced with their motivation and precise definitions to clarify why each component is required for learning an optimal warehouse navigation policy.

Algorithms Used

The problem is solved using a value-based reinforcement learning approach. Specifically, a Deep Q-Network (DQN) is employed, which extends classical Q-learning by using a neural network to approximate the action-value function. This is necessary because explicitly storing Q-values in a table becomes inefficient and inflexible as the state space grows.

Justification of the DQN Framework

The warehouse navigation task involves delayed rewards, sequential decision-making, and constraints imposed by directional walls. The agent must evaluate the long-term consequences of actions rather than optimizing immediate rewards. Value-based reinforcement learning addresses this by

learning a function that estimates the expected cumulative future reward of each action.

Tabular Q-learning defines this estimate explicitly for every state–action pair. However, DQN replaces the table with a neural network, enabling generalization across states that share similar structural properties.

Mathematical and Formal Description

The task is modeled as a Markov Decision Process (MDP), defined by the tuple $(S, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$.

State Space (S). Each state $s \in S$ represents the complete configuration of the environment, including the robot’s grid position, the package status, and the destination. The Markov property holds because the next state depends only on the current state and the chosen action.

Action Space ($\mathcal{A}$). The action space consists of discrete actions corresponding to robot movement and task operations (pickup and drop-off). Discrete actions make value-based methods such as Q-learning and DQN particularly suitable.

Transition Model ($\mathcal{T}$). The transition function $\mathcal{T}(s' | s, a)$ defines the probability of reaching state s' after taking action a in state s . In this environment, transitions are deterministic, but the agent does not have explicit access to $\mathcal{T}$ and must learn from experience.

Reward Function ($\mathcal{R}$). The reward function $\mathcal{R}(s, a)$ provides scalar feedback to guide learning. Positive rewards reinforce successful deliveries, while penalties discourage illegal actions and inefficient navigation. This feedback signal is essential for aligning the learned policy with the task objective.

Discount Factor (γ). The discount factor $\gamma \in (0, 1)$ controls the trade-off between immediate and future rewards. A value of γ close to 1 encourages long-horizon planning, which is necessary for successful pickup-and-delivery behavior.

Action-Value Function The action-value function $Q^\pi(s, a)$ is defined as the expected discounted return obtained by taking action a in state s and thereafter following policy π :

$$Q^\pi(s, a) = E_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

This function quantifies how desirable each action is in a given state and forms the core learning target in DQN.

Bellman Optimality Equation The optimal action-value function $Q^*(s, a)$ satisfies the Bellman optimality equation:

$$Q^*(s, a) = E \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]$$

This recursive relationship decomposes long-term returns into immediate rewards and future value estimates. It provides the theoretical foundation for Q-learning and motivates the update rules used in DQN.

Function Approximation with Neural Networks In DQN, the action-value function is approximated by a neural network $Q(s, a; \theta)$ with parameters θ . The network takes a state s as input and outputs estimated Q-values for all actions. This approximation enables generalization across similar states and reduces memory requirements compared to tabular methods.

Temporal-Difference Learning Objective The network parameters are learned by minimizing the temporal-difference (TD) error, which measures the discrepancy between current Q-value estimates and Bellman targets:

$$\delta = r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)$$

Here, θ^- denotes the parameters of a target network. Using a separate target network stabilizes learning by preventing rapidly moving targets during optimization.

Loss Function The DQN training objective is defined by the mean squared TD error:

$$\mathcal{L}(\theta) = E[\delta^2] = E \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

Minimizing this loss aligns the network’s predictions with the Bellman optimality equation, ensuring convergence toward an optimal or near-optimal action-value function.

Exploration Strategy An ϵ -greedy policy is employed during training to balance exploration and exploitation. With probability ϵ , the agent selects a random action to explore the environment, and with probability $1 - \epsilon$, it selects the action with the highest estimated Q-value. Gradually decaying ϵ allows the agent to transition from exploration to exploitation as learning progresses.

Implementation

This section describes the practical implementation details of the proposed reinforcement learning system, including the software tools and libraries used, the selected hyperparameters and tuning strategy, and the training setup and computational resources.

Tools and Libraries

The entire system is implemented in **Python**, which provides extensive support for scientific computing and reinforcement learning research. The following libraries and frameworks are used:

- **Gymnasium:** Used to define the custom warehouse environment, including the state transition logic, reward function, and episode termination conditions.
- **PyTorch:** Used to implement the Deep Q-Network, perform automatic differentiation, and optimize network parameters using stochastic gradient descent.
- **NumPy:** Used for numerical operations, state encoding, and buffer management.
- **Matplotlib:** Used to visualize training performance through reward curves and policy execution trajectories.

These tools collectively enable efficient environment simulation, neural network training, and performance evaluation within a unified framework.

Network Architecture

The Q-network is implemented as a fully connected feed-forward neural network. Given a state representation s , the network outputs a vector of Q-values corresponding to all possible actions. The architecture consists of:

- An input layer corresponding to the encoded environment state
- Two hidden layers with ReLU activation functions
- An output layer with dimensionality equal to the size of the action space

This architecture balances representational capacity and computational efficiency and is sufficient for the discrete warehouse navigation task.

Hyperparameters and Tuning Strategy

Key hyperparameters used during training are listed below:

- Learning rate $\alpha = 1 \times 10^{-3}$
- Discount factor $\gamma = 0.99$
- Replay buffer size = 5×10^3 transitions
- Mini-batch size = 64
- Target network update frequency = 10 episodes
- Initial exploration rate $\epsilon_0 = 1.0$
- Final exploration rate $\epsilon_{\min} = 0.05$
- Exploration decay schedule: linear decay over training episodes

Hyperparameters are selected using empirical tuning. Initial values are chosen based on standard DQN practices reported in the literature. Subsequent adjustments are made by monitoring training stability and convergence speed, particularly the smoothness and monotonicity of reward curves.

Exploration and Training Dynamics

During training, the agent follows an ϵ -greedy policy. At each time step, an action is selected randomly with probability ϵ , or greedily according to the learned Q-values with probability $1 - \epsilon$. The exploration rate is gradually decayed according to:

$$\epsilon_t = \max \left(\epsilon_{\min}, \epsilon_0 - \frac{t}{T}(\epsilon_0 - \epsilon_{\min}) \right)$$

where t denotes the current training step and T is the total number of decay steps.

This strategy ensures sufficient exploration in early training while enabling exploitation of learned policies in later stages.

Training Duration and Hardware

Training is conducted over several thousand episodes, with each episode terminating upon successful delivery or after reaching a fixed step limit. The total training time depends on the number of episodes and environment complexity.

All experiments are performed on a CPU-based system with the following specifications:

- Processor: Intel-class multi-core CPU

- Memory: 8 GB RAM

- No dedicated GPU is required due to the modest network size and discrete environment

Despite the absence of specialized hardware acceleration, the DQN agent converges reliably within a reasonable training duration, demonstrating the computational efficiency of the proposed approach.

Results

This section presents the experimental results obtained from training and evaluating the Deep Q-Network (DQN) agent in the warehouse navigation environment. Performance is analyzed using quantitative metrics, reward curves, and qualitative visual comparisons against a random-policy baseline.

Evaluation Metrics

The following metrics are used to evaluate agent performance:

- **Cumulative Episode Reward:** The total reward accumulated in an episode, measuring overall task efficiency.
- **Success Rate:** The proportion of episodes in which the agent successfully picks up and delivers the package.

These metrics collectively capture both learning progress and final policy quality.

Training Curves

This subsection presents three individual training curves that collectively describe the learning dynamics of the DQN agent: cumulative reward per episode, exploration rate (ϵ) decay, and success rate per episode.

Cumulative Reward vs. Episode Figure 1 shows the average cumulative reward obtained per episode during training. At the start of training, rewards are low due to random exploration and frequent invalid actions. As training progresses, the reward increases steadily, indicating improved navigation, valid pickup execution, and successful deliveries.

Exploration Rate (ϵ) Decay Figure 2 illustrates the decay of the exploration parameter ϵ over training episodes. Initially, a high value of ϵ promotes exploration of the state-action space. As ϵ decreases, the agent increasingly exploits the learned Q-values, enabling more consistent and goal-directed behavior.

Success Rate vs. Episode Figure 3 shows the success rate per episode, defined as the proportion of episodes in which the agent successfully completes the pickup-and-delivery task. The increasing trend demonstrates that the learned policy becomes progressively more reliable as training advances.

Quantitative Performance Comparison

The random agent fails to complete the task consistently due to uninformed action selection, whereas the DQN agent reliably achieves successful deliveries with fewer steps.

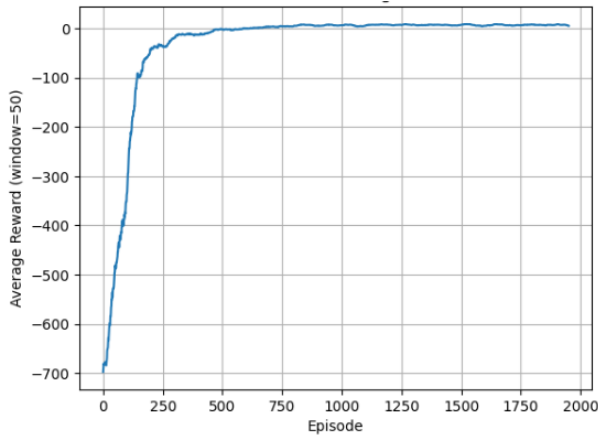


Figure 1: Average cumulative reward per episode during DQN training

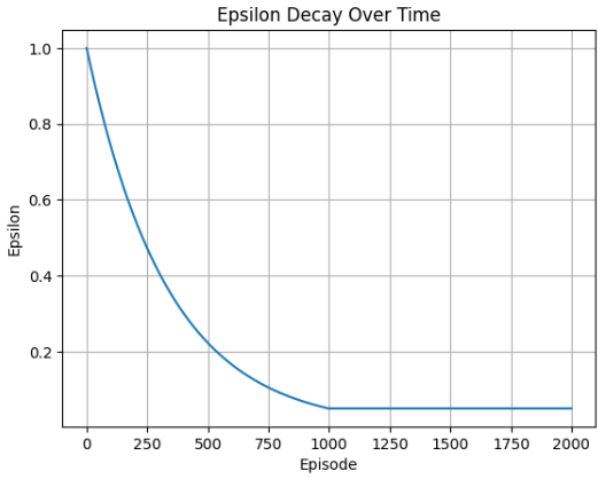


Figure 2: Epsilon decay schedule used during DQN training

Qualitative Visual Analysis

To complement quantitative results, visual snapshots extracted from MP4 rollouts are used to compare agent behaviors. These screenshots highlight the difference between uninformed exploration and learned goal-directed navigation.

Initial State Comparison Figure 4 shows the initial positions of both agents at the beginning of an episode. Both agents start from the same state, ensuring a fair comparison.

Pickup Behavior Figure 5 illustrates the DQN agent successfully reaching the pickup location and executing a valid pickup action. At the same time, the random agent continues to move without task awareness.

Delivery and Goal Completion Figure 6 shows the final stage of the episode. The DQN agent reaches the destination and completes the delivery, whereas the random agent fails to reach the goal within the episode limit.

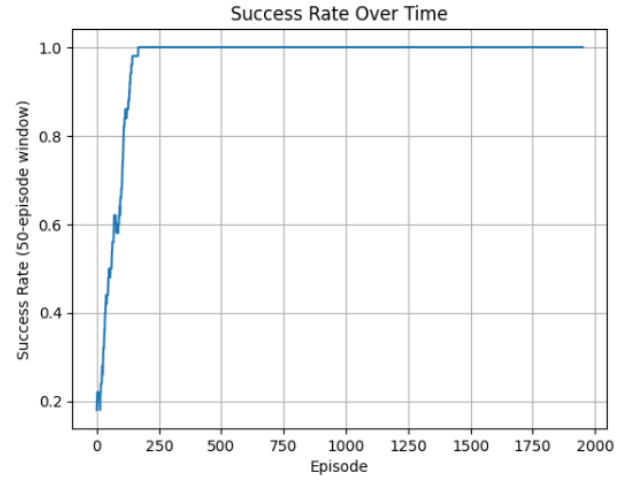


Figure 3: Success rate per episode during DQN training

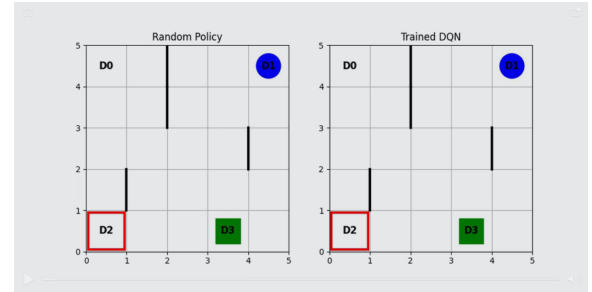


Figure 4: Initial state: both Random and DQN agents start from the same configuration

Result Interpretation

The results clearly demonstrate that the DQN agent learns a meaningful policy that encodes long-horizon planning, obstacle avoidance, and task completion. In contrast, the random policy lacks goal awareness and fails to exploit environmental structure.

The combination of reward curves, performance metrics, and visual evidence confirms that deep reinforcement learning is effective for warehouse robot navigation under movement constraints.

Discussion

This section interprets the experimental results and discusses their implications for warehouse robot navigation using deep reinforcement learning. We analyze the observed learning behavior, highlight unexpected outcomes, and outline the primary limitations encountered during implementation and training.

Interpretation of Results

The training curves and performance metrics demonstrate that the Deep Q-Network (DQN) agent successfully learns a goal-directed navigation policy in the presence of movement

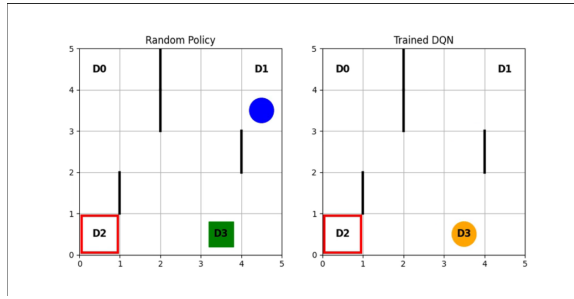


Figure 5: DQN agent successfully picking up the package while the random agent wanders

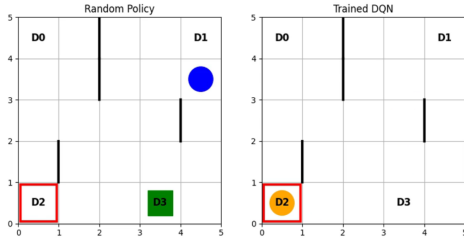


Figure 6: DQN agent delivering the package while the random agent continues wandering

constraints and delayed rewards. The steady increase in cumulative reward and success rate indicates effective policy improvement over time, while the reduction in episode length reflects more efficient path planning.

The comparison with a random-policy baseline further validates the learned behavior. While the random agent fails to consistently perform pickup and delivery actions, the DQN agent reliably completes the task, demonstrating the ability to associate long-term reward with correct action sequences. This confirms that value-based reinforcement learning is well suited for sequential decision-making problems involving structured environments and sparse terminal rewards.

Exploration–Exploitation Dynamics

The epsilon decay curve provides insight into the exploration–exploitation trade-off during training. High exploration in early episodes allows the agent to discover valid transitions and task-relevant states. As ϵ decreases, the agent increasingly exploits learned Q-values, resulting in more stable and consistent task execution.

This transition is critical for convergence, as excessive exploration in later stages can prevent policy refinement, while premature exploitation may lead to suboptimal policies. The observed learning dynamics indicate that the chosen exploration schedule achieves a balanced trade-off.

Anomalies and Unexpected Observations

During early training, sharp fluctuations in cumulative reward are observed. These variations arise from the sparse

nature of terminal rewards and the presence of penalties for invalid actions. Occasionally, the agent discovers a successful delivery sequence by chance, resulting in temporary reward spikes before consistent learning is established.

Another observed behavior is short-term oscillation between nearby states during intermediate training phases. This occurs when multiple actions yield similar Q-values, leading to indecisive behavior until sufficient experience is accumulated. Such oscillations gradually disappear as the Q-value estimates become more accurate.

Limitations and Implementation Challenges

Despite successful learning, several limitations are identified. First, the environment is restricted to a small grid with a single package, which limits the complexity of the task. Scaling to larger warehouses or multiple simultaneous orders would significantly increase the state space and require more advanced architectures or multi-agent reinforcement learning techniques.

Second, the training process is sensitive to hyperparameter selection, particularly the learning rate, discount factor, and exploration schedule. Improper tuning can lead to slow convergence or unstable learning. Careful empirical tuning was required to achieve stable performance.

Finally, training was conducted on CPU-based hardware, which constrained the number of episodes and network complexity that could be explored. While sufficient for this task, larger environments would benefit from GPU acceleration and more extensive experimentation.

Overall, these findings highlight both the effectiveness and the practical challenges of applying deep reinforcement learning to warehouse navigation problems.

Conclusion

This project investigated the application of deep reinforcement learning to autonomous warehouse robot navigation and pickup–delivery tasks. A custom Gymnasium environment with directional movement constraints was designed to model realistic warehouse obstacles and task dependencies. Using a Deep Q-Network (DQN), the agent learned a goal-directed policy that successfully integrates navigation, pickup, and delivery actions under delayed rewards.

The experimental results demonstrate that the project objectives were achieved. The trained DQN agent consistently outperformed a random-policy baseline, as evidenced by higher cumulative rewards, increased success rates, and shorter episode lengths. Training curves confirmed stable convergence, while qualitative visualizations showed that the learned policy exhibits structured, efficient, and purposeful behavior. These findings validate the suitability of value-based deep reinforcement learning for sequential decision-making problems in constrained environments.

Despite these successes, the project also highlights important limitations. The environment is restricted to a small grid with a single package, and training was performed using fixed hyperparameters and CPU-based computation. While sufficient for demonstrating feasibility, these constraints limit scalability and generalization to more complex real-world scenarios.

Several extensions can be explored in future work. The environment can be expanded to larger grids with multiple packages and dynamic obstacles, requiring more advanced state representations. Alternative reinforcement learning algorithms such as Double DQN, Dueling DQN, or policy-gradient methods (e.g., PPO) may further improve stability and sample efficiency. Additionally, multi-agent reinforcement learning could be investigated to model cooperative robot fleets operating in shared warehouse spaces.

Overall, this project demonstrates that deep reinforcement learning provides a principled and effective framework for autonomous warehouse robot control, laying a foundation for more complex and realistic automation systems.

References

- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press, second edition.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.
- Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4):279–292.
- Brockman, G.; Cheung, V.; Pettersson, L.; et al. (2016). OpenAI Gym. *arXiv preprint arXiv:1606.01540*.
- Towers, M.; Kwiatkowski, A.; Terry, J.; et al. (2023). Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2303.00583*.
- Paszke, A.; Gross, S.; Massa, F.; et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Silver, D.; Schrittwieser, J.; Simonyan, K.; et al. (2017). Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359.
- Puterman, M. L. (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley.